Informe: Sistema de Gestión del Torneo Interfacultades de Fútbol 5

Base de Datos 1, Universidad Católica del Uruguay, segundo semestre de 2026

Integrantes: Ianai Canias, Thiago Soca, Gonzalo Muñoz y Sebastián Silveira

Repositorio: https://github.com/Sebastian1891-sys/ObligatorioBD

Fecha: 5 de noviembre de 2026

Introducción

Hicimos un sistema para organizar el Torneo Interfacultades de Fútbol 5 de la UCU: ediciones, equipos, planteles, fixture, planillas, sanciones y reportes. Usamos MySQL 8.0 para la base, Python con FastAPI y PyMySQL para el backend (con el SQL escrito a mano, sin ORM), HTML, CSS y JavaScript puros para el frontend, y Docker Compose para levantar todo con un solo comando.

---

1. Decisiones de implementación

1.1 Dónde vive cada validación

La letra pide validación en las tres capas, y le dimos a cada una un rol distinto. La **base de datos** valida los campos y la integridad con lo que vimos en el curso: `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL` y `CHECK` (formato de cédula y email, rangos de minutos, fechas y camisetas, turnos de una hora). Ningún cliente puede guardar un dato con formato inválido o que apunte a algo que no existe. El **backend** tiene las reglas del torneo que dependen de otros datos (cupos del plantel, cancha libre, rival repetido, suspendidos, ventana de 48 horas, sanciones automáticas) en un único archivo, `reglas.py`: cada regla es una función corta que hace un `SELECT` de chequeo y, si no se cumple, corta con un mensaje en español. Los endpoints las llaman antes de cada `INSERT`, `UPDATE` o `DELETE`, dentro de la misma transacción, así que si algo falla se hace rollback y no queda nada a medias. El **frontend** evita los errores obvios antes de enviar y oculta las acciones que el rol o el estado del partido no permiten.

En una primera versión habíamos puesto las reglas del torneo en triggers y procedimientos de MySQL. Las pasamos al backend porque así quedan todas juntas, se leen como código común (`if`, `for` y consultas simples) y son más fáciles de mantener y de probar. La base sigue protegiendo la integridad de los datos, y los permisos por usuario de MySQL (sección 1.5) limitan qué puede hacer cada rol aunque alguien se conecte por fuera de la aplicación. El backend también traduce los errores de `UNIQUE`, `CHECK` y claves foráneas a mensajes que una persona entienda.

1.2 Reglas resueltas con la estructura de las tablas

Varias reglas de la letra no necesitaron código, sino elegir bien las claves:

- **Un jugador por equipo y edición:** `UNIQUE (ci_jugador, id_torneo)` en `equipo_jugador_torneo`.

- **Ambos equipos de un partido de la misma edición:** claves foráneas compuestas `(id_equipo, id_torneo)` desde `partido` hacia `equipo`.
- **Solo juegan quienes están en el plantel:** `planilla_partido` apunta con clave foránea a `equipo_jugador_torneo`.
- **Solo anotan o reciben tarjetas quienes están en la planilla:** `gol` y `tarjeta` apuntan con clave foránea a `planilla_partido`.
- **Turnos de una hora entre las 18:00 y las 23:00:** dos `CHECK` en `turno`.

1.3 Reglas "salvo que..."

Dos reglas tienen excepción: una cancha no puede tener dos partidos en el mismo día y turno, y dos equipos no pueden enfrentarse dos veces, salvo que el partido haya sido cancelado. Un `UNIQUE` común bloquearía también los partidos que no cuentan, así que las controla el backend: antes de guardar, consulta si ya hay otro partido en esa cancha, día y turno, o entre esos dos equipos (en cualquier orden de local y visitante), sin contar los cancelados. Lo mismo hace con "un partido por equipo y fecha".

Qué hacemos con un partido postergado. La letra no lo define, así que decidimos nosotros:

- Un postergado **libera la cancha**, porque ese horario queda disponible para otro partido.
- Un postergado **sigue contando** para el rival repetido y para "un partido por equipo y fecha". Postergar no es cancelar: el partido se va a jugar, solo que otro día. Si no contara, los mismos equipos podrían terminar jugando dos veces.
- Solo el **cancelado** deja de contar para todo.
- Al reprogramar un postergado, se vuelven a validar todas las reglas, incluida la cancha.

1.4 Sanciones automáticas: recalcular en vez de insertar

Lo primero que pensamos fue insertar una sanción cada vez que llegaba una roja o una tercera amarilla. Lo descartamos porque, si después se corrige o se borra una tarjeta (la letra permite corregir hasta 48 horas después), la sanción ya creada queda colgada. En su lugar, la función `recalcular_sanciones` del backend recalcula desde cero las sanciones automáticas del jugador en la edición, cada vez que se inserta, modifica o borra una tarjeta: primero borra sus sanciones automáticas, después recorre sus tarjetas en orden y vuelve a crear una por cada roja y una por cada tercera amarilla. Así, si el planillero carga una roja por error y la borra, la sanción desaparece sola.

La letra habla de "3 tarjetas amarillas". Nosotros extendimos la regla a cada grupo de tres (la 3.ª, la 6.ª, la 9.ª), para que un jugador no pueda seguir acumulando amarillas sin consecuencias después de su primera sanción. Las sanciones disciplinarias no se recalculan, y el backend no permite editar ni borrar las automáticas: si hay un error, se corrige la tarjeta y la sanción se regenera sola.

1.5 Seguridad en la base

La aplicación nunca se conecta como `root`. En cada request, el backend usa el usuario de MySQL que corresponde al rol de quien inició sesión:

- `torneo_auth`: solo lee la tabla `login`.
- `torneo_admin`: lectura y escritura sobre el esquema, sin poder crear ni borrar tablas.

- `torneo_planillero`: lectura de lo necesario; escritura de planilla, goles, tarjetas y de las sanciones automáticas que se recalculan al cargar tarjetas; y `UPDATE` únicamente de las columnas `estado` y `jugado_en` de `partido`.

Así, aunque hubiera un error en el backend, un planillero no podría borrar un equipo ni cambiar el día de un partido, porque MySQL lo rechaza. Las contraseñas de la aplicación se guardan con bcrypt (costo 10), la sesión es un token firmado con HMAC-SHA256 que vence a las 12 horas, y todas las consultas usan parámetros para evitar la inyección SQL.

1.6 Otras decisiones

- **Resultado calculado, no guardado.** El marcador sale siempre de los goles, mediante vistas (`v_resultado`, `v_tabla_posiciones`, entre otras). Así nunca puede quedar distinto de los goles cargados.
- **Minutos de 0 a 60.** Los turnos duran una hora y la letra no contempla tiempo adicional, así que ese es el rango de goles y tarjetas. Si el reglamento permitiera descuento, solo habría que cambiar un `CHECK`.
- **Reglas extra sobre planillas y tarjetas.** La letra no las menciona, pero las agregamos para evitar datos imposibles: un jugador no puede recibir más de 2 amarillas en un mismo partido, no puede recibir otra tarjeta si ya fue expulsado en ese partido, y no se puede postergar ni cancelar un partido que ya tiene planilla. Las planillas solo se cargan con el torneo en curso. Además, un partido cancelado ya no se puede modificar (hay que programar uno nuevo).
- **Ventana de 48 horas.** La letra dice que el resultado solo se puede corregir dentro de las 48 horas posteriores. Nosotros aplicamos la misma ventana a toda la carga de la planilla: se puede cargar desde que termina el turno y hasta 48 horas después. Lo controla la función `validar_ventana_planilla` del backend, que se llama antes de cargar o corregir cualquier dato de la planilla, los goles o las tarjetas.
- **Datos de prueba generados por un script.** `datos.sql` lo genera `generar_datos.py`, que simula el torneo fecha por fecha con las mismas reglas que el backend. La edición 2026, segundo semestre, queda en curso con 6 equipos de 8 jugadores y 4 fechas disputadas: 11 partidos jugados con 43 goles (uno es autogol) y 14 tarjetas, un partido postergado, uno cancelado y dos programados. Las 3 sanciones son una por roja, una por acumulación de amarillas y una disciplinaria; el script inserta las dos automáticas junto con la tarjeta que las genera, con la misma regla que usa el backend. Además hay una edición 2027 en inscripción con dos equipos incompletos, para probar el ABM de planteles.
- **Docker.** Los scripts SQL se montan en `docker-entrypoint-initdb.d` con prefijos numéricos para que corran en orden, y solo la primera vez que se crea el volumen (por eso `docker compose down -v` devuelve la base al estado inicial). El servicio `app` espera el healthcheck de `db` para no conectarse antes de que MySQL termine de cargar. Los puertos se publican solo en `127.0.0.1`.

---

2. Mejoras al modelo de datos

2.1 Implementadas

Respetamos todas las tablas y atributos de la letra y agregamos lo siguiente:

Dónde	Qué agregamos	Para qué
`login.contrasena`	Guarda un hash bcrypt, con `CHECK` de largo mínimo	Que nunca quede una contraseña en texto plano
`torneo`	`UNIQUE (anio, semestre)`	Una sola edición por semestre
`equipo.ci_capitan`	Admite `NULL`	El equipo se crea antes de tener plantel. Después se asigna el capitán, y el backend exige que sea del plantel
`equipo_jugador_torneo`	`UNIQUE (id_equipo, numero_camiseta)`	No repetir camisetas dentro de un equipo
`partido.jugado_en`	Fecha y hora	Saber cuándo se marcó como jugado
`sancion_jugador.id_tarjeta`	Clave foránea a la tarjeta de origen	Trazabilidad y recálculo de las sanciones automáticas
`sancion_jugador.observacion` y `creada_en`	Un texto y una fecha	Explicar las disciplinarias y dejar un mínimo de auditoría
Vistas	`v_partido`, `v_resultado`, `v_resultado_equipo` y `v_tabla_posiciones`	Calcular resultados y posiciones a partir de los goles

2.2 Consideradas

Estas ideas las discutimos y no las implementamos:

- **Guardar el resultado en `partido`.** Es más rápido de consultar, pero el marcador podría quedar desincronizado de los goles, que es el tipo de error que el sistema quiere evitar. Elegimos la vista, que a esta escala no tiene costo visible.
- **Tabla aparte para las fechas (jornadas).** Descartada: con `numero_fecha` en `partido` alcanza para todas las reglas y reportes, y la tabla extra sumaba un `JOIN` sin aportar nada a lo que pide la letra.
- **Una tabla por tipo de jugador.** Los cuatro tipos tienen los mismos atributos, así que un `ENUM` es más simple y la regla del máximo de 2 docentes o funcionarios se controla igual de bien.
- **Clave sustituta en `planilla_partido`.** Preferimos la clave natural `(id_partido, ci_jugador)`, porque es la que necesitan `gol` y `tarjeta` para que la base haga cumplir que solo anotan y reciben tarjetas quienes están en la planilla.
- **Separar la primera carga de la planilla de su corrección.** Hoy las dos comparten la ventana de 48 horas. Si nadie carga la planilla en ese plazo, el partido queda sin resultado y solo se podría completar directamente en la base. Una alternativa es permitir la primera carga sin límite y aplicar las 48 horas solo a las correcciones.
- **Estado “cumplida” en las sanciones.** Hoy, si se borra una tarjeta que generó una sanción que el jugador ya cumplió, la sanción se recalcula igual. Un estado “cumplida” permitiría congelarla.

- **Tabla de auditoría de planillas.** Registraría quién cambió qué y cuándo. Hoy solo quedan `jugado_en` y `creada_en`.
- **Reglas del torneo también en la base.** Las reglas que controla el backend (cupos del plantel, cancha libre, rival repetido) se podrían repetir con triggers, para que valgan también para alguien conectado por consola. Las dejamos en un solo lugar para no duplicarlas, y los permisos por rol ya limitan qué puede hacer cada usuario de MySQL. El caso que no cubre es el de dos administradores guardando al mismo tiempo dos partidos en la misma cancha y horario; para eso habría que agregar un índice único sobre esas columnas.

---

3. Bitácora

1. **Relevamiento de la letra.** Leímos el enunciado y armamos una lista de todas las reglas del torneo (plantel, programación de partidos, planillas, sanciones, ventana de 48 horas) para decidir cuáles se podían resolver con la estructura de las tablas y cuáles necesitaban lógica.
2. **Diseño del modelo.** Definimos las tablas pedidas y sus claves. Ahí decidimos las claves foráneas compuestas, el `UNIQUE (ci_jugador, id_torneo)` del plantel y que el resultado de un partido se calcule con vistas en lugar de guardarse.
3. **Esquema de la base (`esquema.sql`).** Escribimos la creación de las 14 tablas con sus `CHECK`, índices únicos y claves foráneas, y agregamos los atributos extra (`jugado_en`, `id_tarjeta`, `observacion`, `creada_en`).
4. **Vistas (`logica.sql`).** Armamos las 4 vistas de partidos, resultados y posiciones, que calculan el marcador a partir de los goles, y definimos qué pasa con los partidos postergados.
5. **Usuarios de base de datos (`usuarios.sql`).** Creamos `torneo_auth`, `torneo_admin` y `torneo_planillero` con permisos distintos, incluido el `UPDATE` solo de las columnas `estado` y `jugado_en` para el planillero.
6. **Datos de prueba.** Escribimos `generar_datos.py`, que simula el torneo fecha por fecha respetando las mismas reglas y produce `datos.sql` de forma determinística, con los usuarios de la aplicación ya hasheados con bcrypt. También generamos `script_completo.sql` concatenando los cuatro scripts.
7. **Primera versión del backend.** Armamos la base de FastAPI y PyMySQL (`db.py`, con la conexión por rol y la traducción de errores de MySQL), el login con bcrypt y token firmado, y los ABM de maestros, equipos y planteles. Esta fue la parte que entregamos como avance.
8. **Planillas y sanciones.** Sumamos al backend la carga de planilla, goles y tarjetas, la gestión de sanciones disciplinarias y el avance de estado de las ediciones.
9. **Reportes.** Escribimos las 9 consultas de la letra usando vistas, `JOIN`, `GROUP BY`, `HAVING` y subconsultas, y calculando en Python las posiciones, los totales y los porcentajes. Sumamos las tres propias: jugadores a una amarilla de la suspensión, goles por tramo de 10 minutos y rendimiento como local y visitante.
10. **Frontend.** Hicimos la aplicación de una sola página en HTML, CSS y JavaScript, con las validaciones del lado del cliente (por ejemplo, el dígito verificador de la cédula) y los botones que se ocultan según el rol y el estado del partido.

11. **Pruebas de las reglas.** Probamos cada regla contra la base y la API: planteles, rivales repetidos, canchas ocupadas, suspendidos en planilla, ventana de 48 horas y permisos de cada usuario de MySQL.
12. **Docker y demo.** Armamos el `docker-compose.yml` con los servicios `db` y `app`, y con el healthcheck para ordenar el arranque. Para la defensa preparamos un script que reprograma el partido postergado para ayer, y así poder mostrar la carga de una planilla en vivo.
13. **Documentación.** Escribimos el README con el instructivo para correr la aplicación con y sin Docker, y este informe.
14. **Simplificación.** Revisamos el proyecto contra el material del curso. Pasamos las reglas del torneo de 22 triggers, 3 procedimientos y 3 funciones de MySQL a `reglas.py` en el backend; sacamos las columnas generadas, los `REGEXP` y las funciones de ventana; y volvimos a probar todas las reglas y a comparar los 12 reportes con los de la versión anterior, que dieron igual.

---

4. Bibliografía

- Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of Database Systems* (7.ª ed.). Pearson.
- Silberschatz, A., Korth, H. F. y Sudarshan, S. (2019). *Database System Concepts* (7.ª ed.). McGraw-Hill.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, *13*(6).
- Oracle. *MySQL 8.0 Reference Manual* (CREATE TABLE, CHECK, FOREIGN KEY, CREATE VIEW y GRANT). https://dev.mysql.com/doc/refman/8.0/en/
- FastAPI. *Documentación oficial*. https://fastapi.tiangolo.com/
- PyMySQL. *Documentación oficial*. https://pymysql.readthedocs.io/
- Pydantic. *Documentación oficial (v2)*. https://docs.pydantic.dev/
- Docker. *Compose file reference*. https://docs.docker.com/compose/
- Docker Hub. *mysql: Official Image*. https://hub.docker.com/_/mysql
- OWASP. *SQL Injection Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/
- Material del curso Base de Datos 1, UCU, 2026.